$$\textsc{Float-Const}\ \frac{\mathsf{has}(V, c)}{\Gamma \vdash c : \mathbf{float}[B; V]} \qquad \textsc{Continuous-1}\ \frac{\Gamma \vdash e : \mathbf{float}[B_1; V_1] \quad \mathsf{recovers}(B_1, V_1) \quad \mathsf{dispatchable}(B_1, V_1)}{\Gamma \vdash \mathbf{exponential}(e) : \mathbf{float}[B; \top]}$$

$$\textsc{Continuous-2}\ \frac{\mathsf{recovers}(B_1, V_1) \quad \mathsf{dispatchable}(B_1, V_1) \quad \Gamma \vdash e_1 : \mathbf{float}[B_1; V_1] \quad \Gamma \vdash e_2 : \mathbf{float}[B_2; V_2] \quad \mathsf{recovers}(B_2, V_2) \quad \mathsf{dispatchable}(B_2, V_2)}{\Gamma \vdash \mathbf{uniform}(e_1, e_2) : \mathbf{float}[B; \top]}$$

$$\textsc{Less}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \quad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \quad \mathsf{answers}_{<}(B, V_1, V_2)}{\Gamma \vdash e_1 < e_2 : \mathbf{bool}}$$

$$\textsc{Less-Equal}\ \frac{\Gamma \vdash e_1 : \mathbf{float}[B; V_1] \quad \Gamma \vdash e_2 : \mathbf{float}[B; V_2] \quad \mathsf{answers}_{\leq}(B, V_1, V_2)}{\Gamma \vdash e_1 \leq e_2 : \mathbf{bool}}$$

Fig. 6. Typing rules for float types and continuous distributions (**uniform** is a representative for all other two-argument distributions). The notions *recovers* and *answers* are defined in Definitions 3.2, 3.3 and 3.4. Here Γ is a typing environment of the form $\{x_1 : \tau_1, \ldots, x_n : \tau_n\}$, where $\Gamma, x : \tau_1 = (\Gamma \setminus \{x : \tau \mid \tau \text{ type}\}) \cup \{x : \tau_1\}$.

To understand the typing rules, keep in mind that the cut sets B need to be sufficiently fine-grained — i.e., contain enough cuts — so that the discretized program has enough information to determine how the original continuous program behaved.

Constants. Rule Float-Const assigns a constant c the type $\mathbf{float}[B; V]$ whenever $\mathsf{has}(V, c)$ holds (i.e., $c \in V$ or $V = \top$). Constants do not constrain the cut set, but recording them in V ensures later comparisons know which values must remain distinguishable.

Continuous distributions. Rule Continuous-1 types a single-argument distribution when its argument e has type $\mathbf{float}[B; V]$, $\mathsf{recovers}(B, V)$, and $\mathsf{dispatchable}(B, V)$:

Definition 3.2. Let $B \in \mathcal{B}$ and $V \in \mathcal{V}$. We say that $\mathsf{recovers}(B, V)$ if $B = \top$ or ($V \neq \top$ and every interval I induced by B contains at most one value in V, i.e., $\forall I \in \mathsf{Intervals}(B) : |V \cap I| \leq 1$).

Intuitively, $\mathsf{recovers}(B, V)$ means that the discretization induced by B is lossless on V: from the resulting interval index, the original value in V can be uniquely recovered. This injectivity property is vacuously true for $V = \emptyset$, so distribution parameters also require $\mathsf{dispatchable}(B, V)$, defined by $B = \top$ or $V \notin \{\emptyset, \top\}$. When $B \neq \top$, the value set is therefore finite and nonempty, and B separates its values. The separate dispatchability premise ensures that the distribution-dispatch transformation has at least one case. §5 revisits these invariants. The rule for two-argument distributions is analogous.

Comparisons. For $e < c$ with $e : \mathbf{float}[B_1; \top]$ and $c : \mathbf{float}[B_2; \{c\}]$, the discretized e must still decide the predicate, so B_1 must include $<c$. If c can take any value in V_2, every $<v$ for $v \in V_2$ must appear. Symmetrically, for $c < e$ with $e : \mathbf{float}[B_2; V_2]$, B_2 must contain $\leq c$ so the discretized intervals know on which side of c they fall. For example, $2.3 < e$ requires cuts separating $(-\infty, 2.3]$ from $(2.3, \infty)$; finer partitions are allowed but not required.

In general, the typing rule for $e_1 < e_2$ requires that the expressions share a cut set B, and that their respective value sets V_1, V_2 are related in a way that allows the comparison to be determined after discretization, via the $\mathsf{answers}_{<}(B, V_1, V_2)$ relation:

Definition 3.3. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\mathsf{answers}_{<}(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {<x} \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {\leq x} \in B$.

To understand this definition, we first want to introduce two different perspectives on the B set. The first perspective is that B specifies a set of intervals, such that values of the expression will be mapped to their respective interval index. The second perspective is that this discretized index *still contains enough information to evaluate all comparisons in B when viewed as predicates.* That is, if B contains $<x$ then the discretized value of $e : \mathbf{float}[B; V]$ still contains enough information to answer $e < x$ precisely: if after discretization, e has an interval index below the interval containing x, then the comparison is true, and otherwise it is false.

Now, if $e_1 : \mathbf{float}[B; V_1]$ and $e_2 : \mathbf{float}[B; V_2]$, then $\mathsf{answers}_{<}(B, V_1, V_2)$ ensures that the comparison result can still be determined after discretization because:

(1) If $B = \top$, then no discretization is performed, and the comparison can just be performed on the real-valued expression.
(2) If $B \neq \top$ and $V_2 \neq \top$, then the expression is discretized into intervals as determined by B, but in this case the right-hand side can only take on values in V_2. In this case, all $<x$ for $x \in V_2$ must be contained in B, which means that the value of e_2 can be exactly recovered from its interval index. Then, given the previous observation, we can determine the comparison result by comparing the interval index of e_1 to the interval index of e_2.
(3) If $B \neq \top$ and $V_1 \neq \top$, then the situation is similar to the previous case.

Note that $B = \top$ iff $V = \top$ accounts for comparisons where a continuously ranging expression is compared against arbitrary real values not statically known, such as other continuous distributions, in which case no discretization can be performed. Additionally, we insist that $B_1 = B_2$, which ensures that both comparands will be discretized to the same type, or not discretized at all.

Finally, for non-strict comparisons, the rule is analogous to Definition 3.3:

Definition 3.4. Let $B \in \mathcal{B}$ and $V_1, V_2 \in \mathcal{V}$. We say that $\mathsf{answers}_{\leq}(B, V_1, V_2)$ if (1) $B = \top$, or (2) $B \neq \top$ and $V_2 \neq \top$ and $\forall x \in V_2,\ {\leq x} \in B$, or (3) $B \neq \top$ and $V_1 \neq \top$ and $\forall x \in V_1,\ {<x} \in B$.

Subtyping. The type system includes a subtyping relation that captures when one type can be replaced by another. The subtyping rules are shown in Figure 8. For float types $\mathbf{float}[B; V]$, subtyping follows the lattice ordering on V: an expression known to range over some set of values can also be considered to range over a superset of those values. For B, we require equality to make sure that the discretized representation of the expression is not changed by subtyping.

All other typing rules are standard and depicted in Figure 7 and §D, respectively.

$$\textsc{Var}\ \frac{}{\Gamma, x : \tau \vdash x : \tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1 : \tau_1 \qquad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2}$$

$$\textsc{If}\ \frac{\Gamma \vdash e_1 : \texttt{bool} \qquad \Gamma \vdash e_2 : \tau \qquad \Gamma \vdash e_3 : \tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e : \texttt{bool}}{\Gamma \vdash \texttt{observe}\ e : \texttt{unit}}$$

$$\textsc{Discrete}\ \frac{}{\Gamma \vdash \texttt{discrete}(p_0, \ldots, p_{n-1}) : \texttt{fin}(n)} \qquad \textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \texttt{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1 : \texttt{fin}(n) \qquad \Gamma \vdash e_2 : \texttt{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \texttt{bool}}$$

$$\textsc{FinLessEq}\ \frac{\Gamma \vdash e_1 : \texttt{fin}(n) \qquad \Gamma \vdash e_2 : \texttt{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \texttt{bool}} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \texttt{diverge} : \tau}$$

Fig. 7. Typing rules for general language constructs

$$\textsc{Sub-Refl}\ \frac{\tau \in \{\texttt{int}, \texttt{bool}, \texttt{fin}(n)\}}{\tau <: \tau} \qquad \textsc{Sub-Float}\ \frac{B_1 = B_2 \qquad V_1 \sqsubseteq V_2}{\texttt{float}[B_1; V_1] <: \texttt{float}[B_2; V_2]} \qquad \textsc{Sub-Pair}\ \frac{\tau_1 <: \tau_1' \qquad \tau_2 <: \tau_2'}{\tau_1 * \tau_2 <: \tau_1' * \tau_2'} \qquad \textsc{Sub-List}\ \frac{\tau <: \tau'}{\texttt{list}(\tau) <: \texttt{list}(\tau')}$$

$$\textsc{Sub-Arrow}\ \frac{\tau_1' <: \tau_1 \qquad \tau_2 <: \tau_2'}{\tau_1 \to \tau_2 <: \tau_1' \to \tau_2'} \qquad \textsc{Sub-Type}\ \frac{\tau_1 <: \tau_2 \qquad \Gamma \vdash e : \tau_1}{\Gamma \vdash e : \tau_2}$$

Fig. 8. Subtyping relation. For both cut- and value sets, we let $d \sqsubseteq d'$ iff $\begin{cases} d \subseteq d' & \text{if } d \neq \top \text{ and } d' \neq \top \\ d' = \top & \text{otherwise .} \end{cases}$

$$\begin{aligned} C ::= {} & \tau_1 <: \tau_2 \mid \mathsf{recovers}(B, V) \mid \mathsf{dispatchable}(B, V) \mid \mathsf{answers}_<(B, V_1, V_2) \mid \mathsf{answers}_\le(B, V_1, V_2) \mid \mathsf{has}(V, c) \\ & \mid B_1 \sqsubseteq B_2 \mid B_1 = B_2 \mid V_1 \sqsubseteq V_2 \mid V_1 = V_2 \mid B_1 \equiv_\top B_2 \mid C_1, C_2 \end{aligned}$$

Fig. 9. Syntax for our constraint language. Constraints correspond to the semantic requirements in our type system: subtyping ensures type compatibility, $\mathsf{recovers}(B, V)$ ensures that distribution parameters can be recovered after discretization, $\mathsf{dispatchable}(B, V)$ ensures that a finite dispatch has at least one case, $\mathsf{answers}_<(B, V_1, V_2)$ and $\mathsf{answers}_\le(B, V_1, V_2)$ ensure that comparison results can be determined after discretization, and $\mathsf{has}(V, c)$ ensures that constants are properly tracked.

4 Type Inference

We now describe our type inference algorithm, whose goal is to annotate each subexpression of a program with its most precise type. Specifically, we focus on expressions of type $\texttt{float}[B; V]$ and show how Slice populates the cut set B and value set V, which later determine how and whether the expression can be discretized. Type inference for all other types is standard and follows the classic Hindley-Milner algorithm. For float-typed expressions, we use a two-phase approach: the first phase traverses the program once to generate constraints over cut and value sets; the second phase solves those constraints to produce a fully annotated program.

The distinction between typing and type inference is important. The correctness argument cleanly separates into two pieces: (1) type inference produces a typing derivation matching our typing rules, and (2) given such a typing derivation, discretization is semantics preserving. Thus the proof of discretization does not have to reason directly about the type inference procedure.

4.1 Constraint Generation

We use the constraint language in Figure 9 to express requirements over the type structure and float refinements. We adopt the framework of [38], introducing a new judgment of the form $\Gamma \vdash e : \tau \mid C$, which reads as *in the environment* Γ, e *has type* τ *and produces the set of constraints* C. In this judgment, Γ and e are inputs to the type inference algorithm, while τ and C are outputs: the algorithm infers the type τ for expression e and generates the constraints C that must hold for this typing to be valid. The inference rules formalize which constraints must be generated and later solved for a given expression e to have inferred type τ. Figure 10 gives the constraint generation rules for Slice. We focus on a small but representative subset of constructs in Slice, capturing the key ideas behind constraint generation for the *full* language.

The type inference rules closely mirror the typing rules in Figure 6, with two key differences. First, the inference rules generate a constraint set instead of having those constraints appear in the premises. Second, subtyping is handled in a syntax-directed way: rather than using a general subtyping rule, we bake the subtyping relation into each rule.

Example 4.1. To make the constraint generation process concrete, Figure 11 walks through §2.3. Each subexpression is annotated with type variables for the cut sets (B_i) and value sets (V_i), and the constraints produced at each step are shown explicitly.

4.2 Constraint Solving

Once constraint generation completes, we obtain a system of constraints over type variables, cut sets B, and value sets V. The constraint-solving phase proceeds in two steps:

(1) **Subtyping reduction**: We first process all subtyping constraints of the form $\tau_1 <: \tau_2$. Since subtyping for float types follows the rule $\texttt{float}[B_1; V_1] <: \texttt{float}[B_2; V_2]$ if and only if $B_1 = B_2$ and $V_1 \sqsubseteq V_2$, we reduce each subtyping constraint to corresponding equality and subset constraints on the cut and value sets. Subtyping elimination continues until only constraints among B's and V's remain.

(2) **Lattice constraint solving**: After subtyping reduction, we are left with constraints purely over cut sets and value sets: equalities, subset relations, and the specialized constraints $\mathsf{recovers}(B, V)$, $\mathsf{dispatchable}(B, V)$, $\mathsf{answers}_<(B, V_1, V_2)$, and $\mathsf{has}(V, c)$. These constraints are solved using a monotone fixed-point algorithm over the cut and value lattices.

Subtyping Constraint	After Subtyping Reduction
$\texttt{float}[B_3; V_3] <: \texttt{float}[B_7; V_7]$	$B_3 = B_7,\ V_3 \sqsubseteq V_7$
$\texttt{float}[B_6; V_6] <: \texttt{float}[B_7; V_7]$	$B_6 = B_7,\ V_6 \sqsubseteq V_7$
$\texttt{float}[B_9; V_9] <: \texttt{float}[B_{11}; V_{11}]$	$B_9 = B_{11},\ V_9 \sqsubseteq V_{11}$
$\texttt{float}[B_{10}; V_{10}] <: \texttt{float}[B_{11}; V_{11}]$	$B_{10} = B_{11},\ V_{10} \sqsubseteq V_{11}$

(a) *Subtyping reduction.* Each subtyping constraint over float types is decomposed into an equality constraint on cut sets and a subset constraint on value sets.

	Constraints Processed	Solver Updates
		$\forall i\colon B_i = \emptyset, V_i = \emptyset$
①	$\text{has}(V_1, 0)$	$V_1 := V_1 \sqcup \{0\} = \{0\}$
	$\text{has}(V_2, 2)$	$V_2 := V_2 \sqcup \{2\} = \{2\}$
	$\text{has}(V_4, 0)$	$V_4 := V_4 \sqcup \{0\} = \{0\}$
	$\text{has}(V_5, 1)$	$V_5 := V_5 \sqcup \{1\} = \{1\}$
	$\text{has}(V_8, 1.5)$	$V_8 := V_8 \sqcup \{1.5\} = \{1.5\}$
	$\text{has}(V_9, 1.8)$	$V_9 := V_9 \sqcup \{1.8\} = \{1.8\}$
	$\text{has}(V_{10}, 0.3)$	$V_{10} := V_{10} \sqcup \{0.3\} = \{0.3\}$
②	$V_3 = \top$	$V_3 := \top$
	$V_6 = \top$	$V_6 := \top$
③	$V_3 \sqsubseteq V_7$	$V_7 := V_7 \sqcup \top = \top$
	$V_6 \sqsubseteq V_7$	$\emptyset$
	$V_9 \sqsubseteq V_{11}$	$V_{11} := V_{11} \sqcup \{1.8\} = \{1.8\}$
	$V_{10} \sqsubseteq V_{11}$	$V_{11} := V_{11} \sqcup \{0.3\} = \{0.3, 1.8\}$
④	$\text{recovers}(B_1, V_1),\ \text{dispatchable}(B_1, V_1)$	$\emptyset$
	$\text{recovers}(B_2, V_2),\ \text{dispatchable}(B_2, V_2)$	$\emptyset$
	$\text{recovers}(B_4, V_4),\ \text{dispatchable}(B_4, V_4)$	$\emptyset$
	$\text{recovers}(B_5, V_5),\ \text{dispatchable}(B_5, V_5)$	$\emptyset$
⑤	$\text{answers}_<(B_7, V_7, V_8)$	$B_7 := B_7 \sqcup \{<1.5\} = \{<1.5\}$
	$\text{answers}_\le(B_7, V_7, V_{11})$	$B_7 := B_7 \sqcup \{\le 0.3, \le 1.8\}$ $= \{<1.5, \le 0.3, \le 1.8\}$
⑥	$B_7 = B_8, B_7 = B_{11}$	$B_7 = B_8, B_7 = B_{11}$
	$B_3 = B_7, B_6 = B_7$	$B_3 = B_7, B_6 = B_7$
	$B_9 = B_{10}, B_{10} = B_{11}$	$B_9 = B_{11}, B_{10} = B_{11}$

(b) *Lattice solving.* Each pass performs monotone updates to B_i or V_i. Lines annotated with $\emptyset$ denote constraint is trivially satisfied.

Cut Sets

$B_1 = \emptyset$
$B_2 = \emptyset$
$B_4 = \emptyset$
$B_5 = \emptyset$
$B_3 = B_6$
$= B_7 = B_8$
$= B_9 = B_{10}$
$= B_{11}$
$= \{<1.5, \le 0.3, \le 1.8\}$

Value Sets

$V_1 = \{0\}$
$V_2 = \{2\}$
$V_3 = \top$
$V_4 = \{0\}$
$V_5 = \{1\}$
$V_6 = \top$
$V_7 = \top$
$V_8 = \{1.5\}$
$V_9 = \{1.8\}$
$V_{10} = \{0.3\}$
$V_{11} = \{0.3, 1.8\}$

(c) Final cut sets and value sets obtained.

Fig. 12. Following Figure 11, the constraint solving process for §2.3.

V_2. For compound types, subtyping is handled structurally before reaching floats. For example, a constraint $\texttt{float}[B_1; V_1] * \texttt{float}[B_2; V_2] <: \texttt{float}[B_5; V_5] * \texttt{float}[B_6; V_6]$ is decomposed into $\texttt{float}[B_1; V_1] <: \texttt{float}[B_5; V_5]$ and $\texttt{float}[B_2; V_2] <: \texttt{float}[B_6; V_6]$, each of which is then reduced to equality and subset constraints on the corresponding cut and value sets. This process is syntactic and continues until no subtyping constraints remain, leaving only constraints over B and V.

Lattice Solving. The lattice solver keeps track of the current contents of all B and V variables. Initially, all B and V variables are set to $\emptyset$. During the solving process, the variables may go up in the lattice to a finite set, or to $\top$. Each of the constraints is handled as follows:

- **Value containment** ($\text{has}(V, c)$): We update V to include the constant c, i.e., $V := V \sqcup \{c\}$. If V is already $\top$, the constraint is trivially satisfied. (1)
- **Top assignment** ($V = \top$): We immediately set V to $\top$, representing that the value set contains all possible float values. (2)
- **Subset constraints** ($B_1 \sqsubseteq B_2$ or $V_1 \sqsubseteq V_2$): We ensure that the contents of the first variable are always a subset of the second. When B_1 or V_1 is updated, we propagate the changes to B_2 or V_2 by taking the least upper bound. (3)
- **Recoverability and dispatchability** ($\text{recovers}(B, V)$ and $\text{dispatchable}(B, V)$): Recoverability ensures that the cut set B can distinguish all values in V. If the cuts do not separate the values in the current solver state, we insert additional cuts to satisfy this condition. There is no canonical way to do this, so we use a heuristic: we insert a cut $<x$ for each value $x \in V$ that is not already separated by existing cuts. Dispatchability separately requires a finite, nonempty V when $B \neq \top$. Because $V = \emptyset$ causes no propagating update, the implementation

discharges these logical obligations after monotone propagation reaches its fixed point by traversing the typed sampler nodes and validating their final cut and value bags. Every public discretization entry point performs this validation and rejects an empty or infinite dispatch domain instead of falling back to a sampler. If $B = \top$, both constraints are trivially satisfied. (4)

- **Less-than answerability** ($\text{answers}_<(B, V_1, V_2)$): Intuitively, this constraint ensures that B contains enough cuts to answer all possible comparisons of the form "$v_1 < v_2$" where $v_1 \in V_1$ and $v_2 \in V_2$. If $B = \top$, the constraint is trivially satisfied. Otherwise, we again have a heuristic choice: we add a cut $<v$ for each value $v \in V_2$ (if $V_2 \neq \top$) and $\leq v$ for each value $v \in V_1$ (if $V_1 \neq \top$). If V_1 and V_2 are both $\top$, we set B to $\top$. (5)
- **Less-equal answerability** ($\text{answers}_\leq(B, V_1, V_2)$): Analogous to less-than. (5)
- **Equality constraints** ($B_1 = B_2$ or $V_1 = V_2$): We unify the variables, treating them as the same variable throughout the solving process. Any update to one variable is immediately reflected in all variables unified with it. (6)

The solver repeatedly processes constraints until reaching a fixed point, where no constraint causes any further updates to the variables. The monotonicity of the lattice operations ensures that this process terminates. This procedure may add more cuts to Bs than strictly necessary, but it never sets $B = \top$ unless that is the only way to satisfy all constraints. In other words, the discretization itself may be suboptimal in terms of the number of cuts.

Lattice constraint solving takes time polynomial in the size of the program: at worst, *every* constant in the program gets added to the cut set or value set of every subexpression of type $\texttt{float}[B; V]$.

Additional constraint for contextual equivalence. When the return type of the program involves a float type, either directly or as part of a larger compound type (e.g., $\texttt{float}[B; V] \to \texttt{float}[B'; V']$), we add an additional constraint to ensure that these float types are not discretized. We recursively traverse the return type and set all nested B sets to $\top$. This ensures that the observable behavior of the program is preserved after discretization, regardless of the context in which the program is used. When the return type is basic, such as **bool**, nothing needs to be done.

Example 4.2. To make the constraint solving process concrete, Figure 12 walks through §2.3. Each constraint obtained from the generation phase is processed by the solver, then collected to obtain the final cut and value sets.

5 Discretization

This section presents local transformation rules for our type-directed discretization algorithm, with examples. Our goal is to compute, from a typed expression e : **bool** and its type annotations, a *discretized* expression $\mathcal{D}[\![e : \texttt{bool}]\!] : \texttt{bool}$ such that e and $\mathcal{D}[\![e : \texttt{bool}]\!]$ are *semantically equivalent*, i.e., they induce the same distribution. We refer to this semantics-preserving transformation as *exact discretization*, to distinguish it from approaches [20, 28] that approximate continuous programs via externally chosen precision parameters. Exactness here is the equality proved by Theorem 6.1: the transformed program preserves the relevant probabilities of the source program, not merely an approximation with an error bound. Intuitively, when a program uses continuous quantities only through finitely many comparison outcomes, exact inference should reduce to a discrete problem, and exact discretization makes that reduction explicit.

As shown in Figure 13, $\mathcal{D}[\![e : \tau]\!]$ is defined recursively on $e : \tau$ and its type annotations; we focus on continuous uniform distributions, with others handled analogously. The only sub-expressions replaced by $\mathcal{D}[\![\cdot]\!]$ are float constants and sampling instructions. The rule schemata are local, but their selection and parameters come from globally inferred cut/value information; §6 proves that this non-local, type-mediated construction preserves semantics. Before detailing the rules, we outline how type annotations guide discretization. The type system is designed so that every typed sub-expression of the form $e = \texttt{uniform}(e_1, e_2)$ of type $\texttt{float}[B; V]$ with $B = \{\sim_1 b_1, \ldots, \sim_n b_n\} \neq \top$ can be *discretized* to an expression e' : $\texttt{fin}(n+1)$ such that the probability that e evaluates to a value in $\text{Intervals}(B)_i$ *coincides* with the probability that e' evaluates to $i_{\#n+1}$. In particular, if *all* $\texttt{float}[B; V]$-typed subexpressions are finite-cut, then the given expression can be *fully discretized*.

Example 5.1. Consider a uniform distribution whose outcomes are compared to constants:

```
let x = uniform(0, 1) : float[B; T] in
  (x : float[B; T] < (0.3 : float[B; {0.3}])) ||
   (x : float[B; T] >= (0.5 : float[B; {0.5}]))
```

where $B = \{<0.3, <0.5\}$. The cut set B informs us that the probability of the comparisons evaluating to true is determined by the probability of x evaluating to one of the intervals

$$\text{Intervals}(B)_0 = (-\infty, 0.3) \qquad \text{Intervals}(B)_1 = [0.3, 0.5) \qquad \text{Intervals}(B)_2 = [0.5, \infty)$$

induced by B. Hence, all expressions of type `float[B; ·]` can be discretized to expressions of the finite type $\texttt{fin}(3)$, where $k_{\#3}$ represents the k-th interval $\text{Intervals}(B)_k$ induced by B for $k \in \{0, 1, 2\}$.

Now consider the discretized expression obtained from computing $\mathcal{D}[\![e : \texttt{bool}]\!]$, given by

```
let x = discrete(0.3, 0.2, 0.5) in (x <#3 1#3) || (x >=#3 2#3).
```

The probabilities appearing in **discrete**(...) are determined by the cumulative distribution function (CDF) of the uniform distribution over $[0, 1]$, ensuring that the probability of x evaluating to $k_{\#3}$ in $\mathcal{D}[\![e : \texttt{bool}]\!]$ *coincides* with the probability of x evaluating to a value in $\text{Intervals}(B)_k$ in e. Hence, the constants 0.3 and 0.5 in e can be replaced by the constants $1_{\#3}$ and $2_{\#3}$ in $\mathcal{D}[\![e : \texttt{bool}]\!]$.

Crucially, the *answers* constraint in the premise of the typing rules for comparisons (cf. Figure 6) ensures that B is a sufficiently fine-grained partition of $\mathbb{R}$, i.e., there is an injective function $\text{ToInts}\colon \{0.3, 0.5\} \to \text{Intervals}(B)$ that maps each constant $c \in \{0.3, 0.5\}$ to the unique interval in $\text{Intervals}(B)$ containing c. This interval is determined by the cut convention: a cut $<c$ puts c in the interval to the right of the boundary, while a cut $\leq c$ puts c in the interval to the left.

Example 5.2. If the arguments to a uniform sampling are non-constant, the situation becomes more challenging. Consider the following typed expression e : **bool** (again only showing relevant types):

```
(* Returns true if y is in [lo,hi) *)
let within = fun lo -> fun hi -> fun y ->
  (y >= lo) && (y < hi) in

(* Function for sampling interval bounds *)
let b = fun _ ->
  discrete(0.18: 0.05, 0.11: 0.24,
           0.21: 0.37, 0.09: 0.58,
           0.41: 0.89) in

(* Initialize intervals by sampling their
   respective (possibly overlapping) bounds *)
let init =
  (false, within 1 (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) (b ())) ::
  (false, within (b ()) 8) ::
  nil in

(* Update flags depending on y falling
   within the respective interval *)
let update = fix update y := fun ys ->
  match ys with
    | nil -> nil
    | h :: t ->
        ((fst h || ((snd h) y)), snd h)
        :: (update y t)
  end in

(* Ensure sampled y hits at most one
   interval *)
let non_overlap =
  fix non_overlap y := fun seen -> fun ys ->
    match ys with
      | nil -> true
      | h :: t ->
          let hit = (snd h) y in
          if hit && seen then false
          else non_overlap y (seen || hit) t
    end in

(* Use the sampled value to update flags *)
let step = fun y -> fun ys ->
  update y ys in

(* Number of samples determined by
   unbounded recursion *)
let cover = fix cover st :=
  if discrete(0.1, 0.9) < 1 then st
  else
    let y = discrete(0.5, 0.0199388,
                     0.0748961, 0.0494739,
                     0.0747339, 0.0632619,
                     0.0309625, 0.0280777,
                     0.158655) in
    let _ = observe(non_overlap y false st) in
    cover (step y st) in

(* Have all intervals been covered
   at least once? *)
let is_covered = fix is_covered ys :=
  match ys with
    | nil -> true
    | h :: t -> (fst h) && (is_covered t)
  end in

(* Run the process and query whether
   the whole interval was covered *)
let final = cover init in
  is_covered final
```

Fig. 22. A *discretized* infinite stochastic process for random interval covering over $[0, 1]$, involving conditioning, continuous sampling, higher-order recursion, and lists.

Case $e = (\texttt{if}\ e_1 : \texttt{bool}\ \texttt{then}\ e_2 : \tau\ \texttt{else}\ e_3 : \tau)$. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \tau_i]\!] : \mathcal{D}[\![\tau_i]\!]$ for $i = 1, 2, 3$, where $\tau_1 = \texttt{bool}$ and $\tau_2 = \tau_3 = \tau$. Since $\mathcal{D}[\![\texttt{bool}]\!] = \texttt{bool}$, applying If gives

$$\mathcal{D}[\![\Gamma]\!] \vdash \texttt{if}\ \mathcal{D}[\![e_1 : \texttt{bool}]\!]\ \texttt{then}\ \mathcal{D}[\![e_2 : \tau]\!]\ \texttt{else}\ \mathcal{D}[\![e_3 : \tau]\!] : \mathcal{D}[\![\tau]\!].$$

Case $e = (e_1 : \texttt{float}[B; V_1] < e_2 : \texttt{float}[B; V_2])$. The Less rule was applied, so both subexpressions share the same cut set B and $\text{answers}_<(B, V_1, V_2)$ holds. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \texttt{float}[B; V_i]]\!] : \mathcal{D}[\![\texttt{float}[B; V_i]]\!]$ for $i = 1, 2$.

- If $B = \top$: then $\mathcal{D}[\![\texttt{float}[B; V_i]]\!] = \texttt{float}$ for both i, and the standard float comparison gives type $\texttt{bool} = \mathcal{D}[\![\texttt{bool}]\!]$.
- If $B \neq \top$: then both subexpressions are discretized to type $\texttt{fin}(|B| + 1)$, and the finite comparison $<_{\#(|B|+1)}$ is well-typed at $\texttt{bool} = \mathcal{D}[\![\texttt{bool}]\!]$ by FinLess.

Case $e = (e_1 : \texttt{float}[B; V_1] \leq e_2 : \texttt{float}[B; V_2])$. Analogous to the $<$ case, using $\text{answers}_\leq(B, V_1, V_2)$ and FinLessEq.

Case $e = (\texttt{uniform}(e_1 : \texttt{float}[B_1; V_1],\ e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B; \top])$. The Continuous-2 rule was applied, so $\text{recovers}(B_i, V_i)$ and $\text{dispatchable}(B_i, V_i)$ hold for $i \in \{1, 2\}$. Hence, whenever $B_i \neq \top$, dispatchability gives $V_i \notin \{\emptyset, \top\}$. By the inductive hypothesis, $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e_i : \texttt{float}[B_i; V_i]]\!] : \mathcal{D}[\![\texttt{float}[B_i; V_i]]\!]$ for $i = 1, 2$. We consider each sub-case of the discretization function:

Distribution	Density	Distribution	Density
Exponential	$\lambda e^{-\lambda x}$	Uniform	$\frac{1}{b-a}$
Laplace	$\frac{1}{2b} e^{-\frac{\lvert x-\mu\rvert}{b}}$	Gaussian	$\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$
Cauchy	$\frac{1}{\pi\gamma\left[1+\left(\frac{x-x_0}{\gamma}\right)^2\right]}$	Beta	$\frac{x^{\alpha-1}(1-x)^{\beta-1}}{B(\alpha,\beta)}$
Student-T	$c\left(1+\frac{x^2}{\nu}\right)^{-\frac{\nu+1}{2}}$	LogNormal	$\frac{1}{x\sigma\sqrt{2\pi}} e^{-\frac{(\ln x-\mu)^2}{2\sigma^2}}$
Chi-squared	$\frac{1}{2^{k/2}\Gamma(k/2)} x^{k/2-1} e^{-x/2}$	Gamma	$\frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$
Logistic	$\frac{e^{-(x-\mu)/s}}{s(1+e^{-(x-\mu)/s})^2}$	Pareto	$\frac{\alpha x_m^\alpha}{x^{\alpha+1}}$
Rayleigh	$\frac{x}{\sigma^2} e^{-x^2/(2\sigma^2)}$	Weibull	$\frac{k}{\lambda}\left(\frac{x}{\lambda}\right)^{k-1} e^{-(x/\lambda)^k}$
		Gumbel-1	$\frac{1}{\beta} e^{-(z+e^{-z})}, \quad z=\frac{x-\mu}{\beta}$
		Gumbel-2	$\frac{\alpha\beta x^{-\alpha-1} e^{-(x/\beta)^{-\alpha}}}{\beta}$
		Exponentiated Power	$k\lambda x^{k-1} e^{-(x/\lambda)^k}$
Discrete Distributions			
Poisson	$\frac{\lambda^k e^{-\lambda}}{k!}$	Geometric	$(1-p)^{k-1}p$
Binomial	$\binom{n}{k} p^k (1-p)^{n-k}$	Negative Binomial	$\binom{k+r-1}{k}(1-p)^r p^k$
Hypergeometric	$\frac{\binom{K}{k}\binom{N-K}{n-k}}{\binom{N}{n}}$	Bernoulli	$p^k(1-p)^{1-k}$
Discrete Uniform	$\frac{1}{n}$	Multinomial	$\frac{n!}{x_1!\cdots x_k!} p_1^{x_1}\cdots p_k^{x_k}$

$$\textsc{Var}\ \frac{}{\Gamma, x:\tau \vdash x:\tau} \qquad \textsc{Let}\ \frac{\Gamma \vdash e_1:\tau_1 \quad \Gamma, x:\tau_1 \vdash e_2:\tau_2}{\Gamma \vdash \texttt{let}\ x = e_1\ \texttt{in}\ e_2 : \tau_2} \qquad \textsc{If}\ \frac{\Gamma \vdash e_1:\texttt{bool} \quad \Gamma \vdash e_2:\tau \quad \Gamma \vdash e_3:\tau}{\Gamma \vdash \texttt{if}\ e_1\ \texttt{then}\ e_2\ \texttt{else}\ e_3 : \tau}$$

$$\textsc{Discrete}\ \frac{}{\Gamma \vdash \texttt{discrete}(p_0,\ldots,p_{n-1}) : \texttt{fin}(n)} \qquad \textsc{LessEq}\ \frac{\Gamma \vdash e:\texttt{int}}{\Gamma \vdash e \le i : \texttt{bool}} \qquad \textsc{Pair}\ \frac{\Gamma \vdash e_1:\tau_1 \quad \Gamma \vdash e_2:\tau_2}{\Gamma \vdash (e_1,e_2):\tau_1 * \tau_2} \qquad \textsc{Fst}\ \frac{\Gamma \vdash e:\tau_1 * \tau_2}{\Gamma \vdash \texttt{fst}\ e:\tau_1}$$

$$\textsc{Snd}\ \frac{\Gamma \vdash e:\tau_1 * \tau_2}{\Gamma \vdash \texttt{snd}\ e:\tau_2} \qquad \textsc{Fun}\ \frac{\Gamma, x:\tau_1 \vdash e:\tau_2}{\Gamma \vdash \texttt{fun}\ x \to e : \tau_1 \to \tau_2} \qquad \textsc{App}\ \frac{\Gamma \vdash e_1:\tau_1 \to \tau_2 \quad \Gamma \vdash e_2:\tau_1}{\Gamma \vdash e_1\ e_2 : \tau_2} \qquad \textsc{True}\ \frac{}{\Gamma \vdash \text{true} : \texttt{bool}} \qquad \textsc{False}\ \frac{}{\Gamma \vdash \text{false} : \texttt{bool}}$$

$$\textsc{And}\ \frac{\Gamma \vdash e_1:\texttt{bool} \quad \Gamma \vdash e_2:\texttt{bool}}{\Gamma \vdash e_1\ \&\&\ e_2 : \texttt{bool}} \qquad \textsc{Or}\ \frac{\Gamma \vdash e_1:\texttt{bool} \quad \Gamma \vdash e_2:\texttt{bool}}{\Gamma \vdash e_1\ ||\ e_2 : \texttt{bool}} \qquad \textsc{Not}\ \frac{\Gamma \vdash e:\texttt{bool}}{\Gamma \vdash \text{not}\ e : \texttt{bool}} \qquad \textsc{Unit}\ \frac{}{\Gamma \vdash () : \texttt{unit}}$$

$$\textsc{FinConst}\ \frac{0 \le k < n}{\Gamma \vdash k_{\#n} : \texttt{fin}(n)} \qquad \textsc{FinLess}\ \frac{\Gamma \vdash e_1:\texttt{fin}(n) \quad \Gamma \vdash e_2:\texttt{fin}(n)}{\Gamma \vdash e_1 <_{\#n} e_2 : \texttt{bool}} \qquad \textsc{FinLessEq}\ \frac{\Gamma \vdash e_1:\texttt{fin}(n) \quad \Gamma \vdash e_2:\texttt{fin}(n)}{\Gamma \vdash e_1 \le_{\#n} e_2 : \texttt{bool}} \qquad \textsc{Observe}\ \frac{\Gamma \vdash e:\texttt{bool}}{\Gamma \vdash \texttt{observe}\ e : \texttt{unit}}$$

$$\textsc{Seq}\ \frac{\Gamma \vdash e_1:\tau_1 \quad \Gamma \vdash e_2:\tau_2}{\Gamma \vdash e_1; e_2 : \tau_2} \qquad \textsc{Fix}\ \frac{\Gamma, f:\tau_1 \to \tau_2, x:\tau_1 \vdash e:\tau_2}{\Gamma \vdash \text{fix}\ f\ x := e : \tau_1 \to \tau_2} \qquad \textsc{Nil}\ \frac{}{\Gamma \vdash \text{nil} : \texttt{list}(\tau)} \qquad \textsc{Cons}\ \frac{\Gamma \vdash e_1:\tau \quad \Gamma \vdash e_2:\texttt{list}(\tau)}{\Gamma \vdash e_1 :: e_2 : \texttt{list}(\tau)}$$

$$\textsc{Match}\ \frac{\Gamma \vdash e:\texttt{list}(\tau_1) \quad \Gamma \vdash e_1:\tau_2 \quad \Gamma, h:\tau_1, t:\texttt{list}(\tau_1) \vdash e_2:\tau_2}{\Gamma \vdash \text{match}\ e\ \text{with nil} \to e_1 \mid h :: t \to e_2\ \text{end} : \tau_2} \qquad \textsc{Diverge}\ \frac{}{\Gamma \vdash \texttt{diverge} : \tau}$$

Fig. 25. Typing rules for general language constructs

where $p_k = \text{CDF}(b_{k+1}) - \text{CDF}(b_k)$. Each $p_k \in [0,1]$ and $\sum_k p_k = 1$, so this is well-typed at $\texttt{fin}(|B|+1)$ by Discrete.

- $B \neq \top$, $B_1 \neq \top$, $B_2 \neq \top$, and e_1 or e_2 is a variable: the algorithm generates conditionals over all pairs (v_i, w_j), each branch dispatching to a constant parameter in **uniform**. The value sets V_1 and V_2 are nonempty by the preceding dispatchability argument, so this enumeration has at least one pair and its final else-branch is defined. By the previous sub-case, each branch $\mathcal{D}[\![\texttt{uniform}(v_i, w_j) : \texttt{float}[B;\top]]\!]$ is well-typed at $\texttt{fin}(|B|+1) = \mathcal{D}[\![\texttt{float}[B;\top]]\!]$. The conditional structure is well-typed by If (as in the **if** case above), so the full expression is well-typed at $\mathcal{D}[\![\texttt{float}[B;\top]]\!]$.

$e : \tau$	constraints	$[\![e : \tau]\!]$
$e_1 : \tau_1$ && $e_2 : \tau_2 : \texttt{bool}$ $\tau_1 = \texttt{bool}, \tau_2 = \texttt{bool}$	$e_1 = b_1 = \text{true}, e_2 = b_2 = \text{true}$ $e_1 = b_1, e_2 = b_2$ $e_1 = b_1, e_2 \neq b_2$ o.w.	$\delta_{\text{true}:\texttt{bool}}$ $\delta_{\text{false}:\texttt{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{b_1:\tau_1 \&\& g:\texttt{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \&\& e_2:\tau_2:\texttt{bool}}$
$e_1 : \tau_1$ || $e_2 : \tau_2 : \texttt{bool}$ $\tau_1 = \texttt{bool}, \tau_2 = \texttt{bool}$	$e_1 = b_1 = \text{false}, e_2 = b_2 = \text{false}$ $e_1 = b_1, e_2 = b_2$ $e_1 = b_1, e_2 \neq b_2$ o.w.	$\delta_{\text{false}:\texttt{bool}}$ $\delta_{\text{true}:\texttt{bool}}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{b_1:\tau_1 \| g:\texttt{bool}}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{g \| e_2:\tau_2:\texttt{bool}}$
$\text{not } e : \tau : \texttt{bool}$ $\tau = \texttt{bool}$	$e = \text{true}$ $e = \text{false}$ o.w.	$\delta_{\text{false}:\texttt{bool}}$ $\delta_{\text{true}:\texttt{bool}}$ $[\![e : \tau]\!] \gg\!= \lambda g.\delta_{\text{not } g:\texttt{bool}}$
$(e_1 : \tau_1, e_2 : \tau_2) : \tau_1 * \tau_2$	$e_1 = v_1, e_2 = v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{(v_1,v_2):\tau_1 * \tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{(v_1:\tau_1, f):\tau_1 * \tau_2}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{(f, e_2:\tau_2):\tau_1 * \tau_2}$
$\texttt{fst}\ (e : \tau_1 * \tau_2) : \tau_1$	$e = (v_1, v_2)$ o.w.	$\delta_{v_1:\tau_1}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\texttt{fst } e:\tau_1}$
$\texttt{snd}\ (e : \tau_1 * \tau_2) : \tau_2$	$e = (v_1, v_2)$ o.w.	$\delta_{v_2:\tau_2}$ $[\![e : \tau_1 * \tau_2]\!] \gg\!= \lambda g.\delta_{\texttt{snd } e:\tau_2}$
$(e_1 : \tau_1 \to \tau_2\ e_2 : \tau_1) : \tau_2$	$e_1 = \texttt{fun}\ x \to e$ or $\texttt{fix}\ g\ x := e, e_2 = v_2$ $e_1 = \texttt{fun}\ x \to e$ or $\texttt{fix}\ g\ x := e, e_2 \neq v_2$ o.w.	$\delta_{e[v_2/x]:\tau_2}$ $[\![e_2 : \tau_2]\!] \gg\!= \lambda g.\delta_{e_1:\tau_1 \to \tau_2\ g:\tau_2}$ $[\![e_1 : \tau_1 \to \tau_2]\!] \gg\!= \lambda g.\delta_{f\ e_2:\tau_1:\tau_2}$
$\text{nil} : \texttt{list}(\tau)$		$\delta_{\text{nil}:\texttt{list}(\tau)}$
$(e_1 : \tau_1 :: e_2 : \texttt{list}(\tau)) : \texttt{list}(\tau)$	$e_1 = v_1, e_2 = v_2$ $e_1 = v_1, e_2 \neq v_2$ o.w.	$\delta_{v_1 :: v_2:\texttt{list}(\tau)}$ $[\![e_2 : \texttt{list}(\tau)]\!] \gg\!= \lambda g.\delta_{v_1:\tau_1 :: g:\texttt{list}(\tau)}$ $[\![e_1 : \tau_1]\!] \gg\!= \lambda g.\delta_{f :: e_2:\texttt{list}(\tau):\texttt{list}(\tau)}$
$(\texttt{match}\ e : \texttt{list}(\tau_1)\ \texttt{with}$ $\quad \mid \text{nil} \to e_1 : \tau_2$ $\quad \mid h :: t \to e_2 : \tau_2$ $\quad \texttt{end}) : \tau_2$	$e = \text{nil}$ $e = v_h :: v_t$ o.w.	$\delta_{e_1:\tau_2}$ $\delta_{e_2[v_h/h, v_t/t]:\tau_2}$ $[\![e : \texttt{list}(\tau_1)]\!] \gg\!= \lambda g.\delta_{\texttt{match } g\ \texttt{with} \ldots:\tau_2}$

Table 2. Small-step operational semantics (continued). Here, every function of the form $f(g) = \lambda g.\delta_{\ldots}$ is extended to $f_\perp$ in the sense of Definition F.29.

F.4 Small-Step Semantics

Throughout this section, we denote by $e : \tau$ a well-typed expression of type τ and implicitly assume that all sub-expressions are annotated with types, as well. Moreover, for the typed expression $e : \tau = \texttt{uniform}(e_1 : \texttt{float}[B_1; V_1], e_2 : \texttt{float}[B_2; V_2]) : \texttt{float}[B, V]$ with $B, B_1, B_2 \neq \top$ and $V_1, V_2 \notin \{\emptyset, \top\}$, we introduce an auxiliary statement $\texttt{DC}_{B,B_1,B_2,V_1,V_2}$ and let

$$\mathcal{D}[\![e : \tau]\!] = \texttt{DC}_{B,B_1,B_2,V_1,V_2}(\mathcal{D}[\![e_1 : \texttt{float}[B_1; V_1]]\!], \mathcal{D}[\![e_2 : \texttt{float}[B_2; V_2]]\!]) .$$

We define the semantics of $\texttt{DC}_{B,B_1,B_2,V_1,V_2}$ in such a way that it behaves like the discretized program in the last case of Figure 13 but resolves the let-bindings and the conditional choices in a single step, which will simplify our proof. We equip this statement with the following typing rule :

$$\textsc{TDiscreteCase}\ \frac{e_1 : \texttt{fin}(|B_1| + 1) \qquad e_2 : \texttt{fin}(|B_2| + 1) \qquad \text{recovers}(B_1, V_1) \qquad \text{recovers}(B_2, V_2)}{\texttt{DC}_{B,B_1,B_2,V_1,V_2}(e_1, e_2) : \texttt{fin}(|B| + 1)}$$

Equipped with these notions, we define the small-step semantics of our language:

Definition F.34. The small-step semantics $[\![e : \tau]\!] \in \mathcal{D}(Expr_\tau + \perp)$ is defined recursively on the structure of $e : \tau$ by the rules in Table 1 and Table 2.

The missing probability mass in $[\![e : \tau]\!]$ is the probability of encountering an error due to ill-formed parameters fed into distributions (e.g., $\texttt{uniform}(0.8, 0.1)$). $[\![e : \tau]\!](\perp)$ is the probability of encountering an observation failure.

Lemma F.35 (Measurability of constant extension map). *Let $A \subseteq \mathbb{R}^n$ be equipped with the subspace σ-algebra*

$$\mathcal{F}_A := \{A \cap M \mid M \in \mathcal{B}(\mathbb{R}^n)\},$$